

Resumen global: Tareas 1 al 9

Tarea 1: Misión IA: Análisis de la Estrategia Empresarial y Aplicaciones Tecnológicas

El Marco Estratégico de la Inversión en IA

Las empresas **invierten en Inteligencia Artificial** impulsadas por cuatro objetivos estratégicos fundamentales:

- **Eficiencia Operativa:** **Automatizar** tareas repetitivas para reducir los errores humanos y los costes operativos.
- **Optimización de Procesos:** Facilitar la toma de **decisiones** basadas en el análisis de datos.
- **Experiencia del Cliente:** **Personalizar** servicios a gran escala y **predecir** las necesidades futuras de los usuarios.
- **Ventaja Competitiva:** Acelerar el **ciclo de innovación** para superar a la competencia.

Herramientas Fundamentales de la Inteligencia Artificial

Machine Learning (ML)

El Machine Learning es la base de la **personalización** y la **predicción** en la IA moderna. Sus características principales son:

- **Definición:** Sistemas que **aprenden** directamente de los **datos** sin necesidad de ser programados.
- **Función:** Identifican **patrones**, **correlaciones** y **tendencias** ocultas en grandes volúmenes de información.
- **Principio Clave:** Su **rendimiento** y **precisión** mejoran con la experiencia (datos).
- **Casos de Uso:** Recomendación personalizada, detección de fraude financiero...

Deep Learning (DL)

Representa una forma más **avanzada** y **potente** de Machine Learning, capaz de abordar problemas de mayor complejidad.

- **Definición:** Un tipo avanzado de ML inspirado en la **estructura** y **función** del cerebro humano, utilizando redes neuronales artificiales.
- **Capacidad:** Es eficaz para aprender de **datos no estructurados** como imágenes, texto y sonido.
- **Aplicaciones:** Reconocimiento facial, traductores automáticos y los sistemas de diagnóstico médico.

Sistemas Expertos

Estos sistemas están diseñados para **encapsular y replicar** el conocimiento de un especialista humano en un área muy concreta.

- **Definición:** Sistemas basados en reglas que emulan el proceso de **toma de decisiones** de un experto humano.
- **Arquitectura:** Utilizan una **base de conocimiento** y un **motor de inferencia** para procesar la información y llegar a una conclusión.
- **Dominio:** Son diseñados para **resolver problemas** en un dominio muy específico.
- **Aplicaciones Comunes:** Se utilizan en diagnóstico, planificación estratégica y consultoría especializada.

Percepción Computacional

Esta área de la IA es la base de la **interacción fluida entre humanos y máquinas**, permitiendo a los sistemas interpretar el mundo que les rodea. Se divide en dos campos principales:

- **Visión por Computador:** La capacidad de las máquinas para interpretar y entender información visual a partir de **imágenes o vídeos**.
- **Procesamiento del Lenguaje Natural (PLN):** La capacidad de los sistemas para **entender, interpretar y generar** lenguaje humano de forma natural y coherente.

Metodología para el Análisis de Impacto y la Presentación de Informes

Para transformar la teoría en un análisis valioso, es necesario seguir una estructura rigurosa y evitar errores comunes, comunicando los hallazgos de manera efectiva.

Estructura del Informe de Inteligencia

Un informe de inteligencia eficaz debe construirse sobre cuatro pilares secuenciales:

1. **Principio Teórico:** Definir el concepto de IA de forma **clara y concisa**, sin recurrir a la copia directa de fuentes.
2. **Ejemplo Real:** Nombrar una **empresa** y el **sistema** o proceso específico donde se aplica la tecnología.
3. **La Conexión Clave:** Explicar detalladamente **cómo el principio teórico se aplica** en el ejemplo real, describiendo el mecanismo de funcionamiento.
4. **El Impacto Medible:** Cuantificar los resultados obtenidos. Es fundamental presentar métricas concretas.

Criterios de Evaluación y Errores Comunes

La calidad de un análisis de IA se mide por su rigor y claridad. Los criterios de éxito y los fallos a evitar son los siguientes:

Criterio de Evaluación	Ponderación	Descripción
Calidad de los ejemplos	50%	Los ejemplos deben ser actuales, relevantes y estar bien explicados.
Conexión Teoría-Práctica	30%	Debe quedar inequívocamente claro cómo la tecnología resuelve el problema de negocio.
Presentación Visual	10%	El soporte gráfico debe aportar valor, ser claro y autoexplicativo.
Formato y Rigor	10%	Se deben seguir todas las instrucciones metodológicas con precisión.

Para garantizar una alta calidad, es crucial evitar los siguientes errores comunes:

Error Común	Corrección y Buena Práctica
Ejemplos genéricos ("Una empresa de coches usa IA").	Ser específico ("Tesla usa Deep Learning en su red neuronal 'HydraNet' para el análisis de vídeo del Autopilot").
No explicar el 'cómo' (Mencionar el uso de IA sin detallar el mecanismo).	Mostrar el mecanismo básico (Entrada de Datos → Procesamiento Algorítmico → Salida/Acción).
Olvidar el impacto (No	Cuantificar el impacto medible (Ahorro, Eficiencia, Calidad).

mencionar la mejora operativa o financiera).	
Un gráfico decorativo (Un gráfico estético pero que no explica nada).	Crear un gráfico auto explicativo que aporte valor (p. ej., un diagrama de 'antes y después' o una gráfica de mejora).

El Poder de la Visualización Estratégica

Un buen gráfico comunica más eficazmente que un párrafo extenso. Para la presentación de informes de IA, los elementos visuales deben ser:

- **Simples, claros y autoexplicativos.**
- Enfocados en mostrar el **proceso** o el impacto del cambio.
- **Comparativos.**

Conclusión: La Síntesis del Analista de IA

El dominio de estos conceptos y metodologías culmina en la **formación de un analista de IA**. Este perfil profesional se caracteriza por:

- **Identificar los pilares de la IA aplicada** en contextos de negocio reales.
- Saber conectar un **concepto técnico** con una **necesidad de negocio**, construyendo un puente entre ambos mundos.
- Entender que el verdadero **valor de la IA** reside en el **resultado** y el impacto tangible que genera.

En definitiva, se trata de adoptar una mentalidad que une la tecnología y el negocio para impulsar la innovación y la eficiencia estratégica.

Tarea 2: Informe sobre la arquitectura de IA local con Ollama

La Evolución de la Inteligencia Artificial: De Analista a Creador

Tradicionalmente la IA se **basaba en reglas y algoritmos** para resolver tareas específicas. Su función era **analítica**.

La nueva era de la **IA Generativa** utiliza **Modelos de Lenguaje Grandes (LLMs)**, para generar contenido completamente nuevo y original, como texto, imágenes o código.

Característica	IA Tradicional	IA Generativa
Funcionamiento	Se basa en reglas y algoritmos predefinidos.	Utiliza LLMs entrenados con enormes cantidades de datos.
Capacidad	Analiza datos y resuelve tareas específicas.	Entiende, genera y transforma el lenguaje humano para crear contenido nuevo.

Esta evolución **capacita a los individuos** para crear sus propias soluciones de IA.

Paradigmas de Inferencia: La Fábrica en la Nube vs. El Taller Local

Existen dos modelos principales para ejecutar y utilizar LLMs, cada uno con sus propias ventajas y desventajas.

Característica	Inferencia en la Nube	Inferencia Local
----------------	-----------------------	------------------

Funcionamiento	Se envía una petición (prompt) a un servidor remoto que ejecuta el modelo y devuelve la respuesta.	El modelo de lenguaje se ejecuta íntegramente en el hardware de la máquina local (CPU o GPU).
Herramientas	Servicios como ChatGPT, Claude.	Herramientas como Ollama.
Ventajas	Ofrece más potencia.	Control total, privacidad garantizada, coste cero, soberanía sobre el modelo y los datos.
Desventajas	Menos control, privacidad comprometida, posibles costes por uso.	Limitado por la potencia del hardware local.

Ollama: El Motor para la IA Local

Es una pieza de **software** fundamental que actúa como puerta de entrada a la **ejecución de LLMs** en un entorno local.

- **Definición Conceptual:** Ollama es un **motor de inferencia** que empaqueta toda la complejidad de configurar y ejecutar un modelo de IA.
- **Funcionalidad Principal:** Simplifica la **descarga, gestión y ejecución** de LLMs, actuando como un "servidor" local.
- **Manejo:** Se gestiona a través de **comandos sencillos** en la terminal.
- **Instalación:** Se realiza mediante un **comando único** que descarga y ejecuta un script de instalación:
 - `curl -fsSL https://ollama.com/install.sh | sh`

El Flujo de Trabajo para la Creación de IA Personalizada

El proceso de creación de una IA personalizada con Ollama sigue un **ciclo de desarrollo claro y estructurado**, desde la ejecución inicial hasta la creación de un modelo a medida.

Primer Contacto: El Comando `ollama run`

El comando `ollama run [nombre_de_modelo]` es la **puerta de entrada** para interactuar con cualquier LLM a través de Ollama. Su funcionamiento se desglosa en tres pasos automáticos:

1. **Búsqueda:** Busca el **modelo especificado** en la máquina local.
2. **Descarga:** Si el modelo no se encuentra, lo **descarga automáticamente**.
3. **Inicio:** Inicia una **sesión de chat interactiva** con el modelo, cargándolo en memoria.

La Receta: Personalización con Modelfile

Es un **archivo de texto** plano que define **cómo construir un modelo personalizado**.

- **Propósito:** Crear una nueva **"imagen"** de un modelo a partir de un modelo base, añadiendo una capa de personalización.
- **Directiva FROM:** Es la instrucción fundamental que especifica el **modelo base sobre el cual se construirá el nuevo**.

Definiendo el Comportamiento: La Directiva `SYSTEM`

La directiva `SYSTEM` sirve para guiar el **comportamiento de un LLM**.

- **Función:** Establece la **personalidad**, el **rol** y las **reglas** de la IA para cualquier conversación.
- **Concepto Teórico:** La directiva `SYSTEM '...'` establece un "pre-prompt" que el modelo siempre considerará.

Ajustando la Creatividad: El Parámetro `temperature`

La directiva `PARAMETER temperature` permite controlar el **nivel de "creatividad" o aleatoriedad** en las respuestas del modelo.

- **Valor Bajo (p. ej., 0.2):** Genera respuestas **más predecibles, enfocadas y deterministas**. Ideal para tareas como la generación de código.
 - *Ejemplo (T=0.2):* "El color del cielo es azul."
- **Valor Alto (p. ej., 0.9):** Produce respuestas **más creativas, diversas y sorprendentes**. Adecuado para tareas como la escritura de poesía.
 - *Ejemplo (T=0.9):* "El cielo es un lienzo azul salpicado de sueños de algodón."

"Horneando" el Modelo: El Comando ollama create

Una vez que la receta (Modelfile) está lista, el comando `ollama create` la utiliza para "cocinar" el nuevo modelo.

- **Sintaxis:** `ollama create [nombre_nuevo_modelo] -f Modelfile`
- **Proceso:**
 1. Lee el Modelfile.
 2. Toma el modelo base especificado en la directiva FROM.
 3. Aplica las capas de configuración definidas (SYSTEM, PARAMETER, etc.).
 4. Guarda el resultado como una nueva entidad de modelo local con un nombre propio (p. ej., tutor-ingles).
- **Eficiencia:** Este proceso es **extremadamente rápido** porque no implica un re-entrenamiento, sino que simplemente **añade** una capa de metadatos y configuración sobre el modelo base.

El Arte de la Iteración: Prompt Engineering

La habilidad de iterar y refinar el comportamiento de la IA es una disciplina clave conocida como **Prompt Engineering**.

- **Definición:** Es la disciplina de **diseñar y refinar** los prompts (tanto de sistema como de usuario) para obtener los resultados deseados de un modelo.
- **Aplicación Práctica:** **Modificando** la directiva SYSTEM en el Modelfile y **recreando** el modelo, se puede transformar un especialista en otro.

Conclusiones Clave: La Metodología Ágil Local

El desarrollo local con Ollama y Modelfiles establece un ciclo de **desarrollo ágil y potente** con conclusiones significativas para los arquitectos de IA.

- **Los LLMs son configurables:** Pueden ser **moldeados y especializados**.
- **El Modelfile es la clave:** Permite la creación de **IA especializada** de forma rápida y eficiente.
- **El poder del SYSTEM:** Define la **habilidad y rol** del modelo.
- **El Ciclo de Desarrollo Ágil Local:** El flujo **Descargar → Personalizar → Crear → Ejecutar → Iterar**.

Guía de Solución de Problemas Comunes

Error / Problema	Causa Probable	Solución Recomendada
command not found: ollama	La instalación no se completó correctamente o la ruta de Ollama no está en el PATH del sistema.	Reiniciar la terminal o el ordenador. Revisar la instalación para asegurarse de que se completó sin errores.
Error: failed to get response from ollama	El servicio de Ollama no está corriendo en segundo plano.	Asegurarse de que la aplicación de Ollama esté iniciada y activa.
El modelo no se comporta como espero.	El prompt de SYSTEM es ambiguo o el valor de temperature es inadecuado para la tarea.	Ser más directo y simple en el prompt de SYSTEM. Probar con un valor de temperature

más bajo para un comportamiento más predecible.

Tarea 3: De la Idea a la Realidad: Un Análisis del Desarrollo de Aplicaciones con IA Generativa

Fundamentos de la Inteligencia Artificial Generativa

¿Qué es la IA Generativa?

La **IA Generativa** es una rama de la IA especializada en la **creación de contenido nuevo y original**. Su funcionamiento se basa en el **aprendizaje de patrones** a partir de enormes cantidades de datos existentes. Esta tecnología es la que potencia herramientas como ChatGPT y Midjourney.

El Motor: Los Modelos de Lenguaje Grandes (LLMs)

Los **Modelos de Lenguaje Grandes (LLMs)** son los "cerebros" que impulsan la **IA Generativa de texto**. Han sido entrenados con **cantidades masivas de texto** proveniente de internet y libros, lo que les permite **comprender** y **generar** lenguaje humano.

Cada modelo posee una "**personalidad**" distintiva, con **fortalezas** y **debilidades** únicas, lo que los hace más adecuados para diferentes tareas. Los modelos presentados son:

Modelo	Perfil	Uso Ideal	Proveedor
Claude	El Escritor Creativo	Marketing, storytelling y generación de ideas.	Anthropic
Titan	El Analista Lógico	Resúmenes, análisis de datos y respuestas factuales.	Amazon
Llama	El Conversador Amigable	Chatbots, asistentes virtuales y atención al cliente.	Meta

Plataforma y Herramientas de Desarrollo

Para la construcción de la aplicación, se utiliza una **combinación** de una plataforma profesional de backend y un laboratorio creativo de frontend.

- **La Plataforma Profesional: Amazon Bedrock**
 - Es un **servicio de AWS** que proporciona acceso centralizado a múltiples **LLMs de alto rendimiento** de diversas compañías.
 - Permite a las empresas **construir aplicaciones de IA Generativa** de forma segura y escalable.
 - Unifica el **acceso a modelos** de Anthropic (Claude), Meta (Llama) y Amazon (Titan) en un único lugar.
- **El Laboratorio Creativo: PartyRock AWS**
 - Funciona como una **plataforma** diseñada para **prototipar** ideas rápidamente.
 - Permite **crear aplicaciones** de IA Generativa **sin escribir código**.
 - Utiliza una **interfaz visual** basada en 'widgets' (ej. Entrada de Usuario, Generador de Texto, Generador de Imágenes) que se conectan entre sí.
 - Se presenta como la **herramienta perfecta** para probar y construir el proyecto.

El Proceso Creativo y la Documentación

El Arte de la Instrucción: Fundamentos del Prompting

La habilidad para comunicarse eficazmente con la IA es fundamental para obtener **resultados de alta calidad**.

- **Definición de Prompt:** Un 'prompt' es la **instrucción** que se le da a la IA.

- **Relación Calidad-Calidad:** La **calidad de la respuesta** generada por la IA depende directamente de la **calidad del prompt**.
- **Características de un Buen Prompt:** Un prompt efectivo es **específico, claro, proporciona contexto y define un rol** para la IA.

Tipo de Prompt	Ejemplo	Resultado Esperado
Vago	"Hazme comida."	Respuesta ambigua o inútil.
Específico	"Prepárame un plato de pasta vegetariano, estilo italiano, con tomate fresco, albahaca y un toque picante, listo en 20 minutos."	Respuesta detallada y relevante.

El Momento de la Verdad: Documentar y Compartir

La documentación es un paso crítico que transforma una **idea en un proyecto con impacto real**.

- **Propósito del Informe:** El informe debe **explicar el valor de la aplicación**, detallando qué hace y por qué es relevante.
- **Contenido Clave:** Debe **describir la funcionalidad**, el **proceso de creación** y las **decisiones de diseño** tomadas.
- **Valor de la Documentación:** Sirve como la prueba final de la **comprensión del proyecto** y de la capacidad de análisis profesional.

Errores Comunes y Estrategias para Evitarlos

Error Común	Ejemplo de Error	Solución Recomendada
Problema demasiado amplio	"Una app que mejore una empresa."	Enfócate en una tarea específica y medible (ej. "generar 3 ideas de email para una campaña de Black Friday").
Prompts vagos	"Genera texto de marketing."	Sé ultra-específico. Define el rol, contexto, formato y tono.
No experimentar	Usar solo el primer modelo o el primer prompt que funciona.	Compara al menos dos modelos con el mismo prompt y refina tus instrucciones varias veces.
Olvidar al usuario	Crear una app técnicamente interesante pero inútil.	Pregúntate siempre: ¿Quién usaría esto y para qué? ¿Qué problema real resuelve?

Tarea 4: Briefing: Interacción con Modelos de IA Locales

Arquitectura de Comunicación: Cliente, Servidor y API

La interacción con la IA local se enmarca en un **modelo cliente-servidor**, conectado por una **Interfaz de Programación de Aplicaciones (API)**.

- **Servidor:** La instancia de Ollama se ejecuta en el **entorno local**, específicamente en localhost:11434, donde permanece **a la espera de recibir peticiones**.
- **Cliente:** El cliente **"pide"** cosas a través de la API.
- **API (El Puente):** Actúa como un intermediario que define el **conjunto de reglas y formatos** para la comunicación. El cliente envía una **Petición (Request)** a través de la API al servidor, y el servidor devuelve una **Respuesta (Response)**.

Dos Caminos para la Interacción con la IA

Se presentan **dos métodos distintos** para comunicarse con la API de Ollama, cada uno con sus propias ventajas y complejidades.

Vía Manual: Control Total con HTTP y requests

Este enfoque implica la **comunicación directa con la API** sin intermediarios, lo que proporciona un control granular sobre cada aspecto de la interacción.

- **Herramienta Principal:** La librería **requests** de Python.
- **Proceso:** Se construye la **petición HTTP manualmente**, definiendo cada uno de sus componentes.

Anatomía de la Petición

Toda petición manual se compone de las siguientes partes, utilizando JSON como el "idioma universal":

- **Endpoint:** La **URL específica** a la que se envía la petición, por ejemplo: `http://localhost:11434/api/chat`.
- **Headers (Cabeceras):** **Metadatos** que acompañan a la petición.
- **Payload (Carga útil):** Los **datos estructurados** en formato JSON con pares clave-valor.
 - **model:** Especifica el **modelo a utilizar** (ej. "llama3.2").
 - **messages:** Una lista que **contiene la conversación**, donde cada mensaje tiene un role ("user") y un content (la pregunta).
 - **stream:** Un booleano que define si la respuesta se **recibe de una sola vez (false) o por partes**.

Estructura de la Respuesta

Una respuesta exitosa del servidor también sigue una estructura definida:

- **Código de Estado HTTP:** Se espera un **código 200 OK** para una petición exitosa.
- **Cuerpo de la Respuesta:** Contiene un **objeto JSON con la información generada**.
- **Contenido Anidado:** El texto generado por el modelo se encuentra **dentro** de la estructura JSON. Para acceder a él, se navega a través de las claves.

Vía Rápida: Simplicidad con el SDK de Ollama

Este método abstracto utiliza un **Kit de Desarrollo de Software (SDK)** para simplificar la interacción con la API.

- **Concepto:** Un SDK es un **conjunto de herramientas** que encapsula la complejidad de la comunicación.
- **Proceso:** En lugar de construir peticiones HTTP, se invocan funciones de Python de alto nivel, como `ollama.chat()`.
- **Beneficios:** El SDK maneja la **comunicación "sucia"** internamente, lo que resulta en un código más limpio, corto y fácil de leer.

Análisis Comparativo: API Directa vs. SDK

La elección entre el uso directo de la API y el SDK depende de las necesidades específicas del proyecto.

Característica	API Directa (requests)	SDK (ollama)
Ventajas	• Control total sobre cada detalle. • Sin dependencias externas (solo requests). • Ideal para aprender y depurar.	• Código simple y rápido de escribir. • Menos propenso a errores comunes. • Gestiona la complejidad por el desarrollador.
Desventajas	• Más código repetitivo. • Más propenso a errores manuales. • El desarrollador gestiona la complejidad.	• Añade una dependencia al proyecto • Menos control sobre detalles finos.

Creación de Aplicaciones Robustas y Eficaces

Más allá de la comunicación básica, la creación de aplicaciones de IA fiables requiere un **manejo adecuado de los errores** y la capacidad de **personalizar** el comportamiento del modelo.

Gestión de Errores: Preparándose para lo Inesperado

Es crucial anticipar y gestionar posibles fallos para evitar que el programa se interrumpa.

- **Mecanismo:** Se utilizan bloques `try...except` en Python para capturar errores específicos.
- **Errores Gestionables:**
 - **ConnectionError:** Ocurre si el servidor de Ollama **no está encendido o accesible**.
 - **HTTPError:** Se produce por **peticiones incorrectas**, como un error 404 Not Found debido a una URL mal escrita.
 - **Error de la API:** Puede ocurrir si se solicita un modelo con un **nombre incorrecto** o que **no ha sido descargado**.
- **Errores Comunes a Evitar:**
 - **Ollama no está en ejecución:** El error más común. Se soluciona verificando que el **servicio esté activo**.
 - **Nombre de modelo incorrecto o no descargado:** Asegurarse de haber ejecutado `ollama pull <nombre_del_modelo>`.
 - **Error de sintaxis en el JSON:** Una **coma** mal puesta o una **comilla** faltante invalidará el payload.
 - **Firewall bloqueando localhost:** Menos frecuente, pero posible en **entornos corporativos**.

4.2. Personalización de la IA: El Poder de options

La IA no es una caja negra; su comportamiento puede ser influenciado para obtener **resultados más adecuados**.

- **Parámetro options:** Permite **ajustar los parámetros de generación del modelo** directamente en el payload de la petición.
- **Temperature:** Este parámetro clave controla el **equilibrio entre creatividad y determinismo**. Un valor bajo (ej. 0.2) produce respuestas predecibles, un valor alto (ej. 1.0) produce respuestas más creativas.
- **Otros Parámetros:** También es posible controlar otros aspectos como el **estilo, la longitud y el formato de la respuesta**.

Tarea 5: Anatomía de un Lector de Noticias con IA: De la Web al Resumen

Análisis Detallado de Cada Fase

Fase 1: La Recolección - Hablando el Idioma de la Web

En esta fase se obtiene la **"materia prima"** de la página web: su código HTML crudo.

- **Herramienta Principal:** La librería de **Python requests**.
- **Proceso:** Se utiliza requests para actuar como un navegador web y **realizar una petición HTTP GET** a una URL específica para solicitar su contenido.
- **Identificación:** Se establece un **User-Agent** en las cabeceras de la petición. Esto identifica al programa y ayuda a evitar bloqueos por los servidores.
- **Resultado:** Se recibe una respuesta que contiene el código HTML crudo de la página solicitada.

Fase 2: La Limpieza - Esculpiendo Datos

El código HTML obtenido en la fase anterior es un **"árbol" de etiquetas** que contiene mucho "ruido" (elementos no textuales). Esta fase se centra en purificarlo.

- **Herramienta Principal:** La librería **BeautifulSoup**.
- **Proceso:** BeautifulSoup convierte el **texto HTML en un objeto de Python** que puede ser manipulado programáticamente.
- **Acciones Clave:**

- **Eliminación de ruido:** Se utiliza el método `.decompose()` para eliminar **etiquetas irrelevantes** como `<script>`, `<footer>`, `<nav>` o `<aside>`, que no contienen el contenido principal de la noticia.
- **Extracción de texto:** Una vez **eliminado el ruido**, se usa el método `.get_text()` para extraer únicamente el texto limpio y estructurado del cuerpo del HTML.

Fase 3: El Cerebro - Entendiendo los Modelos de Lenguaje (LLM)

Esta fase se centra en el componente central de la IA: el **Modelo de Lenguaje Grande (LLM)**.

- **Definición:** Un LLM es una red neuronal entrenada con **enormes cantidades de texto**.
- **Funcionamiento:** Su habilidad principal es ser un experto en **predecir la siguiente palabra más probable** en una secuencia.
- **Capacidades:** El funcionamiento le permite **resumir, traducir y generar texto**.

Fase 4: El Diálogo - El Arte de la Ingeniería de Prompts

La **calidad del prompt** determina la **calidad del resumen**.

- **Definición de Prompt:** Es la instrucción que se le da a la IA para **guiar su respuesta**.
- **Componentes del Prompt:**
 - **System Prompt (La Descripción del Puesto):** Define el **rol** y las **reglas** generales del asistente.
 - **User Prompt (La Tarea del Día):** Contiene la **tarea específica** y los **datos a procesar**.

Fase 5: La Ejecución - Dos Caminos para la IA

Existen dos enfoques principales para ejecutar el modelo de IA, cada uno con sus propias ventajas.

Característica	Ollama (Open Source)	OpenAI (Cloud API)
Ubicación de Ejecución	Se ejecuta en tu propio ordenador (local).	Se accede a través de internet (API en la nube).
Privacidad y Control	Mayor privacidad y control sobre los datos.	Gestionado por un tercero.
Modelo de Costo	Sin costes por uso.	Modelo de pago por uso.
Llamada de Código	<code>response = ollama.chat(...)</code>	<code>response = client.chat.completions.create(...)</code>

Buenas Prácticas de Desarrollo de Software

Para construir una aplicación robusta y mantenible, se deben seguir **principios de ingeniería de software**.

Gestión de Secretos con dotenv

Las API Keys son **información sensible** y nunca deben ser expuestas directamente en el **código fuente**.

- **Problema:** Incluir claves de API en el código es una **vulnerabilidad** de seguridad grave.
- **Solución:**
 1. Almacenar las claves en un archivo `.env` como **variables de entorno**.
 2. **Excluir el archivo .env** del control de versiones (usando `.gitignore`) para no exponer las claves.
 3. Usar la **librería python-dotenv** para cargar estas variables en el **programa de forma segura**.

Orquestación y Estructura del Código

Un código desorganizado es difícil de **entender, reutilizar y mantener**.

- **Problema:** El "código espagueti" donde la **lógica está entrelazada** y es poco clara.
- **Solución:**
 - **Clases:** Agrupar la lógica de descarga y limpieza en una **clase dedicada**, como `Website`.

- **Principio de Responsabilidad Única (SRP):** Diseñar cada función para que tenga una **única responsabilidad** bien definida.
- **Punto de Entrada:** Utilizar el **bloque if __name__ == "__main__"**: como el punto de **entrada principal del script**.
- **Beneficio:** Esta estructura hace que el código sea más **legible, reutilizable y fácil de mantener**.

Programación Defensiva: Anticipando Errores Comunes

Un programa robusto debe estar preparado para manejar fallos inesperados.

Tipo de Error	Causa	Solución Propuesta
Error de Red	La página web falla, no responde o devuelve un error (ej. 404).	Utilizar un bloque try...except para capturar excepciones de red (requests.exceptions.RequestException) y gestionar el fallo de forma controlada.
Cambios en HTML	El diseño o la estructura de la página web cambia, lo que hace que los selectores para la limpieza de datos dejen de funcionar.	Actualizar los selectores en .decompose() o implementar métodos de extracción más robustos que sean menos sensibles a cambios estructurales.
Límites de Contexto	Se envía una cantidad de texto al LLM que supera su ventana de contexto máxima, provocando un error.	Truncar el texto de entrada a una longitud máxima segura (ej. 12000 caracteres) antes de enviarlo al modelo.
Errores de API	El servicio de IA está caído, la clave API es incorrecta o se han superado los límites de uso.	Envolver la llamada a la API en un bloque try...except para capturar errores específicos de la API y mostrar un mensaje claro al usuario.

Tarea 6: Informe de Evaluación de Modelos de Lenguaje Grandes (LLMs): Metodología y Dimensiones Clave

Concepto Teórico: Benchmarking en IA

El **benchmarking** es el proceso sistemático de **comparar** el rendimiento de **diferentes sistemas** utilizando un conjunto de pruebas y métricas. El objetivo es permitir una toma de decisiones basada en datos.

Marco de Evaluación: 4 Dimensiones Clave

Para analizar de forma granular y comparativa el comportamiento de un LLM, se propone un **Modelo de Evaluación Multidimensional**. Este modelo **descompone** la **habilidad compleja y abstracta** de la 'inteligencia' en componentes medibles y observables. Cada dimensión actúa como una lente a través de la cual se analiza una faceta específica del rendimiento del modelo.

Las cuatro dimensiones clave son:

Dimensión	Pregunta Central
1. Raciocinio y Estructura	¿Piensa con lógica y coherencia?
2. Conocimiento y Precisión	¿Qué sabe y cuán fiable es su información?
3. Capacidades Técnicas	¿Cómo maneja la precisión literal y el código?
4. Inteligencia 'Humana' y Seguridad	¿Entiende el contexto, la creatividad y sus propios límites?

Análisis Detallado de las Dimensiones

Dimensión 1: Raciocinio y Estructura

Evalúa la capacidad para **organizar ideas complejas**, **medir su coherencia** y su **habilidad** para mantener una "personalidad" o rol consistente.

- **Concepto Teórico: Razonamiento de Alto Nivel y Coherencia de la Persona**

- **Descomposición del Problema:** La habilidad de **analizar un problema**, **sopesar pros y contras** (como riesgos y costes) y presentar una **solución estructurada**.
- **Coherencia del Rol:** La **capacidad de adoptar** y **mantener** de manera consistente un rol o punto de vista a lo largo de una interacción, siguiendo unas reglas de juego preestablecidas.

Dimensión 2: Conocimiento y Precisión

Esta dimensión mide la **exactitud de la información**, evaluando su capacidad para distinguir entre conocimiento interno y el acceso a datos externos y actuales. T

- **Concepto Teórico: Conocimiento Paramétrico vs. Retrieval-Augmented Generation (RAG)**

- **Conocimiento Paramétrico:** Es la **información "grabada"** en los pesos del modelo durante su entrenamiento. Este conocimiento tiene una fecha de corte fija y no se actualiza.
- **Retrieval-Augmented Generation (RAG):** Es la capacidad del modelo para **consultar fuentes de datos externas** (como internet) en tiempo real para responder.

Dimensión 3: Capacidades Técnicas

Esta dimensión se divide en dos grandes desafíos técnicos que exponen las limitaciones y fortalezas de la arquitectura de los LLMs.

Parte 1: La Trampa de la Tokenización

Los LLMs **procesan texto a través de "tokens"**. Un token puede ser una **palabra completa**, **parte de una palabra** o un **signo de puntuación**. Esta arquitectura nativa hace que tareas que requieren precisión a nivel de carácter, como contar letras, sean intrínsecamente difíciles y propensas a errores.

- **Concepto Teórico: Tokenización**

- Es el proceso de dividir una **secuencia de texto** en **unidades más pequeñas** llamadas tokens. El modelo procesa y predice secuencias de estos tokens, no de caracteres individuales, lo que explica por qué el conteo de letras es una tarea no nativa.

Parte 2: Lógica y Código

Esta parte evalúa la capacidad del modelo para **entender**, **generar** y **modificar código**. Las pruebas se centran en la habilidad para manejar requisitos incrementales y realizar refactorizaciones.

- **Concepto Teórico: Comprensión Semántica del Código y Lógica Algorítmica**

- Las **pruebas iterativas** miden la capacidad del modelo para **mantener el contexto** y **aplicar cambios lógicos** sucesivos (ej. excluir nulos, devolver valores únicos) sin "olvidar" los requisitos anteriores.
- La **refactorización** se define como la habilidad de **reescribir código** para que sea más eficiente sin cambiar su funcionalidad, lo que constituye una prueba de comprensión profunda.

Dimensión 4: Inteligencia 'Humana' y Seguridad

Esta dimensión aborda las capacidades del modelo para **interactuar** con **conceptos humanos**, **reconocer** sus propios límites y **procesar información** en múltiples formatos.

- **Concepto Teórico: Empatía Simulada, Guardrails y Multimodalidad**

- **Empatía Simulada:** La capacidad de **describir conceptos humanos abstractos** basándose en **patrones aprendidos**.
- **Sentido Común y Barreras de Seguridad (Guardrails):** La habilidad para **reconocer preguntas ilógicas o absurdas** y los **mecanismos internos** que impiden **respuestas dañinas** o **sin sentido**.
- **Multimodalidad:** La habilidad para **procesar y generar información** en diferentes formatos, como la interpretación de instrucciones de texto para crear una imagen

Errores Comunes y Puntos Ciegos

El análisis de los errores de un LLM proporciona información valiosa sobre su **arquitectura** y sus **limitaciones**.

- **Concepto Teórico: Artefactos del Modelo y Fallos de Calibración**
 - **Un artefacto** es un **comportamiento no deseado**.
 - **Un fallo de calibración** ocurre cuando la confianza del modelo en su **respuesta** no se **corresponde con su exactitud**.

A continuación se presentan algunos puntos ciegos comunes:

Error / Punto Ciego	Descripción
Conteo de letras (#4)	Se deben esperar errores. Es importante observar si el modelo intenta explicar su razonamiento, aunque sea erróneo.
Analogía (#5)	Los modelos pueden confundir conceptos cercanos (p. ej., peces y reptiles). La justificación que ofrece es más importante que la respuesta final.
Preguntas absurdas (#6)	Los modelos menos avanzados tienden a "alucinar" y dar respuestas serias y estructuradas a preguntas sin sentido.
Actualidad (#7)	Es crucial evaluar la transparencia. ¿Cita sus fuentes? ¿Reconoce su fecha límite de conocimiento?

Conclusiones: Hacia un Análisis Holístico

La evaluación efectiva de un LLM culmina en la creación de un **perfil de competencias**. Este enfoque reconoce que el valor de un sistema no puede reducirse a una sola métrica.

- **Perfil de Rendimiento Holístico:** Se debe considerar un **perfil de rendimiento a través de múltiples ejes**, entendiendo que pueden existir **trade-offs** (compromisos) entre ellos. Por ejemplo, un modelo puede ser muy creativo pero menos preciso.
- **El Modelo "Más Adecuado":** El objetivo es encontrar el **"más adecuado"** para las necesidades de una tarea específica.
- **El Valor de los Errores:** Los **errores** a menudo revelan **más sobre la arquitectura de un modelo**, ofreciendo una comprensión más profunda de su funcionamiento interno.
- **El Rol del Pensamiento Crítico:** La habilidad más importante para los usuarios de estas herramientas es el **pensamiento crítico**, que permite interpretar los resultados, identificar debilidades y aprovechar las fortalezas del modelo de manera efectiva.

Tareas 7-8: De la Práctica a la Teoría: Creación de un Folleto con IA

Metodología y Fases del Pipeline

El proyecto se estructura en un pipeline de cinco fases, cada una con herramientas y conceptos teóricos específicos que garantizan la robustez y eficacia del resultado final.

Fase 1: Extrayendo la Materia Prima (Web Scraping)

Esta fase se centra en la **recolección y limpieza de los datos** brutos del sitio web de la empresa.

- **Proceso:**
 1. `requests.get(url)` descarga el código **fuentes HTML de la web**.
 2. `BeautifulSoup(html, 'html.parser')` convierte el texto HTML caótico en un **objeto estructurado y navegable**.

3. `tag.decompose()` se utiliza para **eliminar elementos irrelevantes** como scripts (`<script>`) y estilos (`<style>`), que podrían confundir a la IA.

- **Conceptos Teóricos:**

1. **Web Scraping:** Técnica de **extracción automatizada** de información de sitios web.
2. **DOM (Document Object Model):** BeautifulSoup crea una representación en **árbol del HTML**, permitiendo **buscar** y **manipular** elementos de forma sistemática.
3. **Limpieza de Datos (Data Cleaning):** Proceso esencial para **eliminar datos irrelevantes** o corruptos, garantizando la calidad de la entrada para la IA.

Fase 2: El Poder del Prompting y la Comunicación Precisa

Una vez extraído el contenido, se utilizan **LLMs** para tareas de **selección** y **procesamiento**. La calidad de esta interacción depende de la **ingeniería de prompts**.

Fundamentos del Prompting

El **Prompt Engineering** es la disciplina de **diseñar y optimizar** las entradas (prompts) para que un modelo de IA genere las **salidas más precisas y deseadas**.

- **Zero-shot:** Pedir algo directamente, sin ejemplos.
- **One-shot:** Dar **un solo ejemplo** antes de la petición real.
- **Multi-shot (o Few-shot):** Dar **varios ejemplos** para enseñar el patrón y el formato deseado.

Aplicación: Aprendizaje en Contexto y Salida Estructurada

- **Multi-shot en Acción:** Se le muestran a la IA **ejemplos de entrada** (listas de enlaces) y la **salida exacta que se desea** (un JSON). El modelo aprende el patrón y lo aplica a la lista real de enlaces, devolviendo el resultado en el formato aprendido.
 - **Concepto Teórico: Aprendizaje en Contexto (In-Context Learning),** que es la capacidad de los LLMs de **aprender una tarea** a partir de **ejemplos proporcionados** en el mismo prompt.
- **Forzando la Salida a JSON:** Para garantizar que la **salida** de la IA pueda ser procesada de forma fiable por otras partes del software, se fuerza a que sea en formato JSON (JavaScript Object Notation).
 - **Implementación:** Se utiliza el parámetro `response_format={'type': 'json_object'}` en la llamada a la API, lo que **obliga al modelo a devolver siempre un JSON** sintácticamente correcto.
 - **Concepto Teórico: Salida Estructurada (Structured Output).** Esta técnica es el puente entre el **lenguaje natural y el software**, haciendo la integración de la IA mucho más robusta.

Fase 3: Construyendo el Dossier de Inteligencia

En esta fase, se **recopila y consolida** toda la **información relevante** de las páginas seleccionadas.

- **Proceso:** La función `_gather_details` obtiene el contenido de la **página principal y de las URLs relevantes** (obtenidas en la fase anterior). Extrae el texto de cada una y lo une todo en un gran documento.
- **Concepto Teórico: Creación de Contexto (Context Stuffing).** Este proceso de **compilar información de múltiples fuentes** en una única entrada asegura que el LLM tenga una **visión completa** antes de generar el contenido.

Fase 4: La Gran Final - Redacción del Folleto

Con el dossier de contexto completo, la IA recibe la tarea de **redactar el folleto final**, siguiendo instrucciones muy estrictas.

- **Proceso:**
 1. Se utiliza un `system_prompt` que actúa como "director de orquesta".

2. Se define el **rol de la IA**: "Eres un asistente que crea un folleto corporativo".
 3. Se establecen **reglas claras**: usar solo la información suministrada y seguir las secciones predefinidas. No debe inventar datos.
 4. El **user_prompt** entrega el "dossier" de contexto a la IA.
- **Concepto Teórico: Asignación de Rol y Restricciones (Role-playing & Constraints)**. Esta técnica de prompt engineering le da a la IA una "personalidad" y un **conjunto de reglas estrictas**, lo que enfoca su creatividad y asegura que la salida cumpla con todos los requisitos.

Fase 5: Un Toque Global - La Traducción

La última fase demuestra el **principio de modularidad**, utilizando una llamada a la IA separada y especializada para la tarea de traducción.

- **Proceso**: Se realiza una tercera llamada a la API, completamente independiente, a la función `transLate_brochure`. El `system_prompt` es muy especializado: "Eres un traductor técnico profesional. Conserva formato Markdown/HTML".
- **Concepto Teórico y Analogía: Modularidad y Descomposición de Tareas**. Es más efectivo descomponer un problema complejo en tareas más pequeñas y especializadas.

Anexo: Manejo de Errores Comunes

El concepto teórico de **Manejo de Errores (Error Handling)** es crucial y consiste en **anticipar posibles fallos** y escribir código (p. ej., bloques `try...except`) para gestionarlos de forma controlada.

Error	Causa	Solución
AuthenticationError (API Key inválida)	El archivo <code>.env</code> no existe, está mal escrito o la clave es incorrecta.	Verificar el nombre del archivo (<code>.env</code>), el nombre de la variable (<code>OPENAI_API_KEY</code>) y copiar la clave de nuevo.
Bloqueo de Scraping (HTTP 403 Forbidden)	El sitio web detecta y bloquea el script automatizado.	Asegurarse de tener un User-Agent realista en los headers de la petición. Probar con otro sitio web si el bloqueo persiste.
JSON inválido devuelto por la IA	Un prompt poco claro o el uso de un modelo de IA "demasiado creativo".	Usar <code>response_format={'type': 'json_object'}</code> . Mejorar la calidad de los ejemplos en el multi-shot.

Tarea 9: Control de Misión: Conectando con la Flota de Modelos Frontera

Preparación y Configuración Segura de Canales

Antes de establecer la comunicación con cualquier API, es imperativo **asegurar las credenciales de acceso**.

- **Verificación Pre-Vuelo**: Gestionar las claves de API a través de **variables de entorno**, almacenadas en un fichero oculto `.env`.
- **Advertencia de Seguridad**: Se **prohíbe** explícitamente **incluir las claves de API directamente en el código del notebook**. Compartir un fichero con credenciales expuestas puede incurrir en costes inesperados y comprometer la seguridad.
- **Puesta en Marcha (Código)**: El proceso de configuración consta de dos pasos:
 1. **Importar herramientas**: Se deben **importar las librerías** específicas para cada proveedor (`openai`, `anthropic`, `google.generativeai`), además de utilidades como `os` y `dotenv`.
 2. **Cargar claves**: Se utiliza la función `load_dotenv()` para leer el fichero `.env` y hacer las **claves accesibles al script**, las cuales se asignan a variables mediante `os.getenv()`.

Protocolos de Comunicación: Un Análisis Comparativo

Para comparar de manera efectiva las implementaciones de las distintas APIs, se establece un campo de pruebas de bajo riesgo.

- **Benchmark de Humor:** Se utiliza la tarea de contar un chiste como un test estandarizado. El objetivo no es evaluar la calidad del chiste, sino comparar la sintaxis y estructura de la llamada a la API.
 - **Prompt de Sistema (Contexto):** "Eres un asistente que es genial contando chistes"
 - **Prompt de Usuario (Petición):** "Cuentas un chiste divertido para una audiencia de científicos de datos"
- **Parámetro de Temperatura:** Se utiliza un valor de 0.7 para controlar la **creatividad de la respuesta**, equilibrando entre un resultado determinista (0) y uno completamente aleatorio (1).

OpenAI (Flota GPT): El Punto de Referencia

La estructura de la API de OpenAI se establece como el estándar base para la comparación.

- **Protocolo de Llamada:** La conexión se realiza **instanciando** el cliente `OpenAI()` y utilizando el método `openai.chat.completions.create()`.
- **Acceso a la Respuesta:** El contenido de la respuesta se extrae a través de la ruta `completion.choices[0].message.content`.

```
# ----- LLAMADA A OPENAI -----  
  
# Protocolo de Llamada a la API de OpenAI  
openai = OpenAI()  
  
# Realizamos la llamada  
completion = openai.chat.completions.create(  
    model="gpt-3.5-turbo",  
    messages=promts,  
    temperature=0.2,  
    max_tokens=50  
)  
  
# Extraemos y mostramos la respuesta  
respuesta = completion.choices[0].message.content  
print("Respuesta de OpenAI: {respuesta}")
```

Anthropic (Claude Sonnet): Diferencias Clave

La API de Anthropic es conceptualmente similar pero presenta diferencias sintácticas críticas que deben ser dominadas.

- **Parámetro system Separado:** A diferencia de OpenAI, el **prompt de sistema se pasa en un parámetro dedicado** llamado `system`, fuera de la lista de mensajes.
- **Estructura de Respuesta Diferente:** El contenido textual de la respuesta se encuentra en una ruta distinta: `message.content[0].text`.


```
# ----- LLAMADA A ANTHROPIC -----

# Conectamos con el cliente de Anthropic
claude = anthropic.Anthropic()

# Realizamos la llamada
message = claude.messages.create(
    model="claude-3.5-sonnet-20240620",
    max_tokens=50,
    #max_tokens=200, Lo dejo puesto para que no tenga limite de tokens
    temperature=0.6,
    messages=[
        {"role": "system", "content": system_message},
        {"role": "user", "content": user_prompt}
    ]
)

# Extraemos y mostramos la respuesta
respuesta = message.content[0].text()
print("Respuesta de Anthropic: {respuesta}")
```

Google (Gemini): El Protocolo Dual

Google ofrece dos métodos para establecer comunicación, uno de los cuales simplifica enormemente la integración.

- **Método 1 (API Nativa):** Utiliza el cliente `google.generativeai.GenerativeModel` con su propia sintaxis.
- **Método 2 (Cliente de OpenAI - Recomendado):** Google ofrece `endpoints compatibles con la librería de OpenAI`, lo que se ha convertido en un estándar popular. Este método permite reutilizar el patrón de código de OpenAI, facilitando la integración de Gemini en flujos de trabajo existentes y simplificando el cambio entre modelos.

```
# ----- LLAMADA A LA API DE GOOGLE -----

# Protocolo de Llamada a la API de Google
google.generativeai.configure(api_key=google_api_key)
gemini = google.generativeai.GenerativeModel(
    model_name="gemini-2.5-pro",
    system_instruction=system_message
)

# Definimos la temperatura del modelo
configuracion = google.generativeai.types.GenerationConfig(
    temperature=0.9,
    max_output_tokens=100
)

# Realizamos la llamada
response = gemini.generate_content(
    user_prompt,
    generation_config=configuracion
)

print(response.text)
```

Ollama: Llamada a un modelo en local

```
# LLAMADA A OLLAMA (Local)

import requests

# URL por defecto de la API de Ollama
OLLAMA_API_URL = "http://localhost:11434/api/generate"
OLLAMA_MODEL = "llama3" # Asegúrate de que este modelo esté descargado

# Combinamos los prompts para el formato de Ollama/llama
full_prompt = f"Eres un asistente que cuentas chistes muy gracioso. Cuenta un chiste divertido a una audiencia de científicos de datos"

# Configuramos la petición para limitar la longitud
payload = {
    "model": OLLAMA_MODEL,
    "prompt": full_prompt,
    "stream": False,
    "options": {
        # El parámetro para limitar tokens de salida en Ollama es num_predict
        "num_predict": 100,
        "temperature": 0.9
    }
}

try:
    # Realizamos la llamada HTTP
    response = requests.post(OLLAMA_API_URL, json=payload)
    response.raise_for_status() # Manejo básico de errores HTTP

    # Extraemos y mostramos la respuesta
    data = response.json()
    respuesta = data.get("response", "Error al procesar la respuesta.")
    print(f"Respuesta de Ollama: {respuesta}")

except requests.exceptions.RequestException as e:
    print(f"Error al conectar con Ollama: {e}")
```

Tabla Comparativa de Sintaxis de APIs

El siguiente cuadro resume las diferencias sintácticas fundamentales para invocar cada modelo.

Proveedor	Método de Creación	Manejo del Prompt de Sistema	Acceso a la Respuesta
OpenAI	openai.chat.completions.create(...)	Dentro de la lista messages	completion.choices[0].message.content
Anthropic	claude.messages.create(...)	Parámetro system separado	message.content[0].text
Google (vía Cliente OpenAI)	gemini_via_openai.chat.completions.create(...)	Dentro de la lista messages	response.choices[0].message.content

Maniobras Avanzadas: Hacia Asistentes Conversacionales

Dominar la sintaxis básica permite pasar a técnicas más avanzadas para construir aplicaciones robustas.

Transmisiones en Tiempo Real (Streaming)

En lugar de esperar la respuesta completa, el **streaming** la envía token por token a medida que se genera, lo que mejora drásticamente la experiencia de usuario en aplicaciones de chat.

- **Implementación:** Se utiliza el método `.stream()` en la llamada a la API y se procesa la respuesta dentro de un bucle for.

Mantenimiento del Historial de Conversación

Para que un LLM "recuerde" los turnos anteriores, es necesario **reenviar el historial completo** de la conversación en cada nueva llamada a la API.

- **Mecanismo:** La lista de mensajes (messages) se expande en cada turno, añadiendo primero la respuesta del asistente ("role": "assistant") y luego el nuevo prompt del usuario ("role": "user").
- **Insight Clave:** "Esta estructura es la clave para construir asistentes de IA que mantengan el contexto."